Colas de prioridad y heaps

Notas de clase

Estructuras de datos y algoritmos

Facultad TIC - UPB

Spring 2026

Jorge Mario Londoño Peláez

3 de marzo de 2026

Priority Queues and Heaps

Implementación y análisis de colas de prioridad y estructuras heap.

Universidad Pontificia Bolivariana

Índice

1. Definición, Aplicaciones **2**

2. API de cola de prioridad **2**

3. Árboles binarios completos y montículos **3**

3.1. Definición y propiedades del árbol binario completo 3

3.2. Montículo 3

3.3. Operaciones auxiliares: `swim` y `sink` 3

3.4. Ejemplo de ejecución de `swim` y `sink` 4

3.5. Operaciones de la Priority Queue (PQ) con montículos 5

4. Ordenación por montículo: Heapsort **6**

5. Comparación de implementaciones de colas de prioridad **7**

1. Definición, Aplicaciones

Una **cola de prioridad** es un Abstract Data Type (ADT) que almacena elementos, cada uno con una clave asociada que determina su prioridad. A diferencia de las colas (FIFO) o pilas (LIFO), el orden de extracción se basa en la prioridad de las claves, no en el orden de llegada.

En una **MaxPQ** (cola de máxima prioridad), el elemento con la mayor prioridad se elimina primero.

En una **MinPQ** (cola de mínima prioridad), el elemento con la menor prioridad se elimina primero.

La prioridad determina el orden en que se procesan los elementos.

Ejemplos de aplicación de la cola de prioridad

- **Gestión de tareas en un sistema operativo:** Se usan para priorizar la ejecución de procesos, dando preferencia a los más importantes. Por ejemplo, un proceso del sistema podría tener mayor prioridad que un programa de usuario.
- **Algoritmo de Dijkstra (camino más corto):** Se usa una **MinPQ** para seleccionar el nodo más cercano al origen que aún no se ha visitado.
- **Compresión de datos (algoritmo de Huffman):** Se usa una **MinPQ** para construir el árbol de Huffman, uniendo los nodos con menor frecuencia.
- **Simulación de eventos discretos:** Se usa una PQ para gestionar la secuencia de eventos, procesando primero los eventos con el tiempo de ocurrencia más próximo. Por ejemplo, simular el tráfico en una ciudad.

Ejemplo sencillo: Imagina una sala de emergencias en un hospital. Los pacientes no se atienden en el orden en que llegan, sino según la gravedad de su condición. Los pacientes con problemas más graves (mayor prioridad) se atienden primero.

2. API de cola de prioridad

La API básica de una cola de prioridad (asumiendo una MaxPQ) incluye las siguientes operaciones:

- `insert(key)`: Inserta un nuevo elemento con clave `key` en la cola de prioridad.
- `max()`: Retorna el elemento con la clave máxima (mayor prioridad) en la cola de prioridad, sin eliminarlo.
- `delMax()`: Elimina y retorna el elemento con la clave máxima en la cola de prioridad.
- `changeKey(element, key)`: Cambia la clave de un `element` existente a un nuevo valor `key`. El `element` puede identificarse mediante un índice o una referencia mantenida por una estructura auxiliar. Esta operación es fundamental en algoritmos como Dijkstra, donde la "distancia" o prioridad de un nodo puede cambiar.
- `size()`: Retorna el número de elementos en la cola de prioridad.
- `isEmpty()`: Retorna verdadero si la cola de prioridad está vacía, falso en caso contrario.

En el caso de la cola de mínima prioridad (MinPQ), las operaciones son similares, pero se nombran `min()` y `delMin()`.

3. Árboles binarios completos y montículos

3.1. Definición y propiedades del árbol binario completo

Un **árbol binario completo** es un árbol binario en el que todos los niveles, excepto posiblemente el último (el de las hojas), están completamente llenos, y todos los nodos en el último nivel están lo más a la izquierda posible.

Propiedades:

- Un árbol binario completo con n nodos tiene una altura $h = \lfloor \lg n \rfloor$ [1].
- El número de nodos en un árbol binario completo de altura h está entre 2^h y $2^{h+1} - 1$.

3.2. Montículo

Un **montículo** (o *heap* en inglés) es un árbol binario completo que cumple la **propiedad del montículo**:

- **MaxPQ:** La clave de cada nodo es mayor o igual que las claves de sus hijos. La raíz tiene la clave más grande.
- **MinPQ:** La clave de cada nodo es menor o igual que las claves de sus hijos. La raíz tiene la clave más pequeña.

Los montículos se suelen representar como arreglos para facilitar el cálculo de los índices de los hijos. Se prefiere la representación por medio de arreglo porque resulta más eficiente en términos de espacio (vs. usar objetos Nodo que tienen un overhead adicional).

Indexación con base 1 (convención de este capítulo): Si un nodo está en la posición k, sus hijos estarán en las posiciones $2k$ y $2k + 1$. Si un nodo está en la posición j, su padre está en la posición $j \div 2$ [2].

Indexación con base 0: En lenguajes con arreglos de índice cero (como Java o Python), las fórmulas son diferentes: si un padre está en el índice i, sus hijos están en $2i + 1$ y $2i + 2$. El padre de un nodo en el índice j está en $\lfloor (j - 1)/2 \rfloor$.

3.3. Operaciones auxiliares: `swim` y `sink`

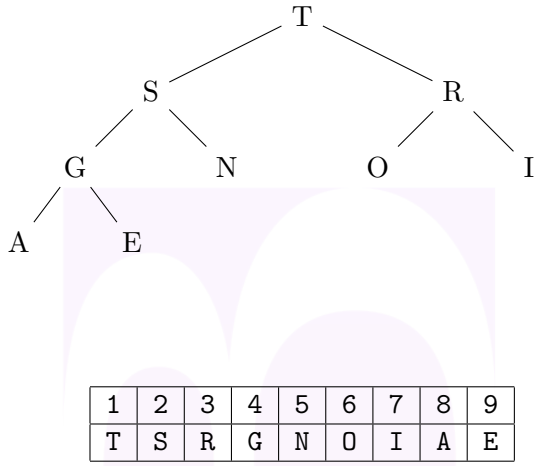
`swim` y `sink` son cruciales para mantener la propiedad del montículo tras insertar o eliminar elementos.

- `swim(k)` (Flotar): Si la clave del nodo en la posición `k` es mayor que la de su padre, se intercambia con el padre. Se repite hasta que la clave del nodo no sea mayor que la de su padre o hasta llegar a la raíz. Se usa tras la inserción o al incrementar la clave de un elemento.
- `sink(k)` (Hundir): Si la clave del nodo en la posición `k` es menor que la de alguno de sus hijos, se intercambia con el hijo mayor (en un MaxPQ). Se repite hasta que la clave del nodo no sea menor que la de sus hijos o hasta llegar a una hoja. Se usa tras la eliminación o al disminuir la clave de un elemento.

[1] La convención $\lg n = \log_2 n$

[2] Se utiliza $\div$ para denotar la división entera, es decir $j \div 2 = \lfloor j/2 \rfloor$.

Figura 1: Representación de un montículo como árbol y como arreglo.



Pseudocódigo de `swim(k)`:

```
procedure SWIM(A, k)
    while k > 1 and A[k] > A[k ÷ 2] do
        Intercambiar A[k] y A[k ÷ 2]
        k ← k ÷ 2
    end while
end procedure
```

Pseudocódigo de `sink(k)`:

```
procedure SINK(A, k, N)
    while 2k ≤ N do
        j ← 2k
        if j < N and A[j] < A[j + 1] then
            j ← j + 1
        end if
        if A[k] ≥ A[j] then
            break
        end if
        Intercambiar A[k] y A[j]
        k ← j
    end while
end procedure
```

Complejidad: Ambas operaciones tienen complejidad $O(\log n)$ porque, en el peor caso, deben recorrer la altura del árbol.

3.4. Ejemplo de ejecución de `swim` y `sink`

Ejemplo de `insert` con `swim`: Consideremos insertar la clave `V` en el montículo `[T, S, R, G, N, O, I, A, E]`:

1. Se añade `V` al final: `[T, S, R, G, N, O, I, A, E, V]`

2. `swim(10)`: `V` en posición 10 compara con su padre `N` en posición 5. Como `V >N`, intercambian.
3. Resultado: `[T, S, R, G, V, O, I, A, E, N]`
4. `swim(5)`: `V` en posición 5 compara con su padre `T` en posición 1. Como `V >T`, intercambian.
5. Resultado final: `[V, S, R, G, T, O, I, A, E, N]`

Ejemplo de `delMax` con `sink`: Consideremos eliminar el máximo del montículo `[V, S, R, G, T, O, I, A, E, N]`:

1. Se intercambia la raíz `V` con el último elemento `N`: `[N, S, R, G, T, O, I, A, E]`
2. Se elimina el último elemento (que contiene `V`). Tamaño del montículo: 9
3. `sink(1)`: `N` en posición 1 compara con sus hijos `S` (pos. 2) y `R` (pos. 3). El mayor es `S`, y como `N <S`, intercambian.
4. Resultado: `[S, N, R, G, T, O, I, A, E]`
5. `sink(2)`: `N` en posición 2 compara con sus hijos `G` (pos. 4) y `T` (pos. 5). El mayor es `T`, y como `N <T`, intercambian.
6. Resultado final: `[S, T, R, G, N, O, I, A, E]`

3.5. Operaciones de la PQ con montículos

Las operaciones de la PQ se implementan eficientemente con montículos binarios completos (representados como arreglos):

- `insert(key)`:
  1. Añade el nuevo elemento al final del arreglo.
  2. Llama a `swim(n)`, donde n es la posición del nuevo elemento.

  Complejidad: $O(\log n)$ por la operación `swim`.
- `max()`:
  1. Devuelve el elemento en la primera posición del arreglo (la raíz del montículo).

  Complejidad: $O(1)$.
- `delMax()`:
  1. Intercambia la raíz (el máximo) con el último elemento del arreglo.
  2. Elimina el último elemento del arreglo (que ahora contiene el máximo).
  3. Llama a `sink(1)` para restaurar la propiedad del montículo desde la raíz.

  Complejidad: $O(\log n)$ por la operación `sink`.
- `changeKey(element, key)`:

1. Se localiza el índice `k` del `element`. Esto puede requerir una búsqueda lineal $O(n)$ o, más eficientemente, una estructura de datos auxiliar (como una tabla hash) que mapee elementos a sus índices en $O(1)$.
2. Se actualiza la clave del elemento en la posición `k` a `key`.
3. Se restaura la propiedad del montículo llamando a `swim(k)` y luego a `sink(k)`. Una de las dos operaciones restaurará el orden, dependiendo de si la clave aumentó o disminuyó.

Complejidad: $O(\log n)$ si se usa una estructura auxiliar para localizar el elemento en $O(1)$. Sin estructura auxiliar, la complejidad total es $O(n)$ debido a la búsqueda lineal. En aplicaciones como el algoritmo de Dijkstra, donde `changeKey` se usa frecuentemente, es esencial usar la estructura auxiliar para mantener eficiencia.

4. Ordenación por montículo: Heapsort

Heapsort es un algoritmo de ordenación que usa montículos. Tiene dos fases:

1. **Construcción del montículo (Heapify):** Transforma el arreglo de entrada en un montículo.
2. **Fase de ordenación:** Extrae repetidamente el elemento máximo del montículo y lo coloca al final del arreglo ordenado.

Construcción del montículo a partir de un arreglo (Heapify):
Se puede transformar un arreglo en un montículo en tiempo lineal $O(n)$ usando el método *heapify*. Se trata el arreglo como un árbol binario completo y se aplica `sink` a todos los nodos no hoja, desde el último hasta la raíz.

Pseudocódigo de `Heapify(A,N)`:

```
procedure HEAPIFY(A, N)
    for k ← N ÷ 2 downto 1 do SINK(A, k, N)
    end for
end procedure
```

El algoritmo `Heapify` tiene una complejidad $O(n)$. Aunque se llama a `sink` (una operación $O(\log n)$) unas $n/2$ veces, un análisis más detallado muestra que el costo total no es $O(n \log n)$.

Análisis de la complejidad de Heapify: La clave está en observar que la mayoría de los nodos están en los niveles inferiores del árbol, donde el costo de `sink` es bajo:

- En un árbol binario completo de altura h, hay a lo sumo $\lceil n/2^{h+1} \rceil$ nodos a altura h.
- Un nodo a altura h puede hundirse a lo sumo h niveles.
- El costo total está acotado por: $\sum_{h=0}^{\lfloor \lg n \rfloor} \lceil n/2^{h+1} \rceil \cdot O(h) = O(n \sum_{h=0}^{\lfloor \lg n \rfloor} h/2^h)$.
- La serie $\sum_{h=0}^{\infty} h/2^h$ converge a 2, por lo tanto el costo total es $O(n)$.

Intuitivamente: la mayoría de los nodos ($n/2$) son hojas y no requieren `sink`, el siguiente cuarto ($n/4$) solo puede hundirse 1 nivel, el siguiente octavo ($n/8$) puede hundirse 2 niveles, etc. Esto hace que el costo promedio por nodo sea constante.

Algoritmo Heapsort:

El proceso de ordenación funciona de la siguiente manera:

1. Se construye un MaxPQ a partir del arreglo de entrada usando *heapify*. El elemento más grande está ahora en la raíz.
2. Se repite el siguiente proceso para i desde N hasta 2:
   a) Se intercambia el elemento de la raíz $A[1]$ (el máximo actual) con el último elemento del montículo, $A[i]$. El elemento máximo queda así en su posición final correcta.
   b) Se reduce el tamaño efectivo del montículo en 1 (excluyendo el elemento ya ordenado).
   c) Se llama a `sink(1)` en el montículo reducido para restaurar la propiedad del montículo en la raíz.

Al finalizar, el arreglo está completamente ordenado en orden ascendente.

Pseudocódigo de Heapsort:

```
procedure HEAPSORT(A, N)
    HEAPIFY(A, N)                          ▷ Construir el montículo: O(n)
    for i ← N downto 2 do
        Intercambiar A[1] y A[i]           ▷ Mover máximo al final
        SINK(A, 1, i − 1)                  ▷ Restaurar propiedad del montículo
    end for
end procedure
```

Propiedades de Heapsort:

- **Complejidad temporal:** $O(n \log n)$ en el peor, promedio y mejor caso. La fase de heapify toma $O(n)$ y la fase de ordenación ejecuta $n-1$ operaciones `sink`, cada una $O(\log n)$.
- **Complejidad espacial:** $O(1)$ de memoria auxiliar (solo necesita variables temporales para intercambios). Es un algoritmo *in-situ*.
- **Estabilidad:** Heapsort **NO es estable**. Los intercambios de largo alcance (entre la raíz y el final del arreglo) pueden cambiar el orden relativo de elementos con claves iguales.
- **Ventajas:** Garantiza $O(n \log n)$ en el peor caso (a diferencia de Quicksort) y no requiere memoria adicional.
- **Desventajas:** En la práctica, suele ser más lento que Quicksort bien implementado debido a la mala localidad de referencia (accesos a memoria dispersos en el montículo).

5. Comparación de implementaciones de colas de prioridad

Existen varias formas de implementar una cola de prioridad, cada una con diferentes trade-offs de rendimiento:

Implementación	**insert**	**max/min**	**delMax/delMin**	**changeKey**
Arreglo desordenado	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Arreglo ordenado	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Lista enlazada desordenada	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Lista enlazada ordenada	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Montículo binario	$O(\log n)$	$O(1)$	$O(\log n)$	$O(\log n)^*$
Árbol binario de búsqueda balanceado	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$

Nota: * Con estructura auxiliar (tabla hash) para localizar elementos en $O(1)$. Sin ella, `changeKey` es $O(n)$.

Recomendaciones:

- **Montículo binario:** Es la implementación más común y eficiente para la mayoría de aplicaciones. Ofrece el mejor balance entre operaciones de inserción, extracción y cambio de clave.
- **Arreglo desordenado:** Solo útil cuando las inserciones son mucho más frecuentes que las extracciones.
- **Arreglo ordenado:** Solo útil cuando las extracciones son mucho más frecuentes que las inserciones (poco común).
- **Árbol balanceado:** Útil cuando se necesita acceso eficiente tanto al máximo como al mínimo simultáneamente, o cuando se requieren operaciones adicionales como búsqueda por rango.

Variantes avanzadas de montículos: Existen estructuras más sofisticadas para casos especiales:

- **Montículo binomial (Binomial Heap):** Soporta unión de dos colas de prioridad en $O(\log n)$.
- **Montículo de Fibonacci (Fibonacci Heap):** Operaciones `insert` y `changeKey` en $O(1)$ amortizado, `delMax` en $O(\log n)$ amortizado. Usado en algoritmos avanzados de grafos.
- **Montículo d-ario:** Generalización del montículo binario donde cada nodo tiene d hijos. Puede ser más eficiente para ciertas aplicaciones.